

Sistem autonom de răspuns la furtunile de praf pentru o aeronavă pe Marte

Partea 1 – Analiza problemei

Întelegerea scenariului

O aeronavă autonomă se află deja în zbor pe Marte atunci când detectează o furtună de praf aflată la aproximativ 12 minute înainte pe ruta curentă. Comunicarea cu Pământul are o întârziere de 14 minute dus-întors, astfel încât aeronava trebuie să decidă autonom ce acțiune execută. La prima vedere, soluția pare simplă, aeronava detectează furtuna și o ocolește. Consider însă că problema reală este mai complexă. Aeronava nu cunoaște starea viitoare exactă a furtunii, ci lucrează cu măsurători și predicții care sunt incomplete și contradictorii. În plus, orice manevră consumă timp și energie și trebuie să respecte limitele fizice ale aeronavei.

Aș formula problema astfel: cum poate aeronava să ia, într-un timp limitat, o decizie de siguranță justificabilă atunci când informațiile despre pericol sunt incerte, comunicarea cu operatorii este prea lentă, iar energia și capacitatea de manevră sunt limitate. Înainte de proiectarea soluției, consider necesară separarea informațiilor cunoscute de presupunerile introduse de mine.

Din cerință știu doar că aeronava este deja în zbor, o furtună de praf este detectată la aproximativ 12 minute înainte, comunicația cu Pământul are aproximativ 14 minute întârziere dus-întors și aeronava trebuie să decidă autonom. Nu știu însă severitatea și dimensiunea furtunii, viteza și direcția ei, precizia estimării de 12 minute, ce senzori au detectat-o și cât sunt de fiabili, dacă orice furtună este periculoasă pentru aeronavă, viteza, altitudinea și raza minimă de viraj a aeronavei, tipul de propulsie, energia disponibilă, energia necesară unei aterizări de urgență, existența unor zone sigure de aterizare, nivelul de autonomie deja disponibil la bord și cât de importantă este continuarea misiunii față de conservarea aeronavei. Aceste necunoscute sunt importante deoarece nu aș vrea ca o soluție să depindă de proprietăți ale aeronavei pe care enunțul nu le menționează.

Înainte de implementare, aș încerca să clarific patru categorii de informații.

- Despre furtună
 - Ce senzori indică existența furtunii?
 - Sunt măsurătorile independente?
 - Există un nivel de încredere asociat detecției?
 - Cum se estimează poziția și deplasarea furtunii?
 - De la ce severitate devine traversarea nesigură?
 - Cum se comportă sistemul dacă senzorii se contrazic?
- Despre aeronavă
 - Care este viteza actuală și raza minimă de viraj?
 - Ce manevre sunt fizic posibile în intervalul disponibil?
 - Ce energie este disponibilă?
 - Ce rezervă este necesară pentru o aterizare sigură?
 - Există aterizare autonomă?
- Despre misiune
 - Există puncte de întoarcere și zone de aterizare cunoscute?
 - Ce întârziere a misiunii este acceptabilă?
 - Cum este definită o stare sigură?
- Despre comunicație
 - Cele 14 minute includ doar propagarea sau întregul ciclu operațional de răspuns?
 - Aeronava poate continua să transmită telemetrie în timpul manevrei?

Problema fiabilității informației

Una dintre cele mai importante observații este că furtună detectată nu ar trebui reprezentată printr-o simplă variabilă booleană. Aeronava observă semnale, de exemplu reducerea vizibilității, creșterea concentrației de particule, schimbări de presiune și modificări ale vântului.

Din aceste observații rezultă: probabil există o furtună severă pe ruta curentă. Prin urmare, aș păstra explicit trei niveluri observații - inferență – încredere în inferență. Această separare este importantă deoarece sistemul nu dispune de un adevăr de referință instantaneu. Dacă se construiește o verificare, trebuie să se analizeze și dacă verificarea însăși este fiabilă. Pentru prototip, aș defini trei stări ale dovezilor, CONFIRMAT (mai multe semnale relevante sunt consistente), INCERT (există indicii, dar încrederea este redusă) și CONFLICT (surse

importante se contrazic). De exemplu, dacă doar camera indică un nivel foarte ridicat de praf, iar ceilalți senzori indică valori normale, nu aș considera automat furtuna confirmată. Sistemul ar trebui să reducă nivelul de încredere, să mărească marja de siguranță și dacă timpul permite, să colecteze informații suplimentare.

Cum validez mecanismul de detecție

Dacă folosesc un prag precum $C > 0,70 \Rightarrow$ furtună confirmată, trebuie imediat pusă întrebarea: de ce 0,70. Aș testa mecanismul prin valori fals pozitive, furtuni reale nedetectate, valori imediat deasupra și dedesubtul pragului, senzori indisponibili, un senzor blocat permanent la valoarea maximă, contradicții între senzori și eliminarea pe rând a câte unui senzor. Scopul nu este doar să verific dacă algoritmul produce un rezultat, ci dacă rezultatul rămâne rezonabil atunci când datele sunt imperfecte. Aceasta este una dintre principalele dificultăți ale scenariului, nu există un answer key disponibil în timpul zborului.

Comparația directă este $12 \text{ min} < 14 \text{ min}$. Prin urmare, un ciclu complet de comunicare cu Pământul nu furnizează o decizie înainte de întâlnirea estimată cu furtuna. Termenul real este chiar mai strict. Aeronava are nevoie și de timp pentru calcul, inițierea manevrei și crearea unei marje de siguranță. Aș defini $T_{limită} = T_{pericol} - T_{manevră} - T_{siguranță}$. Dacă presupun, doar pentru prototip $T_{pericol} = 12 \text{ min}$, $T_{manevră} = 2 \text{ min}$ și $T_{siguranță} = 2 \text{ min}$ rezultă $T_{limită} = 8 \text{ min}$. Deci aeronava nu așteaptă 12 minute pentru a decide. Trebuie să acționeze mult mai devreme.

Așteptarea unor măsurători suplimentare crește nivelul de încredere, dar reduce timpul disponibil pentru manevră. Astfel apare compromisul valoarea informației suplimentare vs. costul întârzierii. Sistemul nu trebuie să aștepte certitudine absolută. Dacă timpul disponibil devine prea mic, o acțiune conservatoare este mai sigură decât continuarea colectării de date.

O rută care evită furtuna nu este neapărat sigură. De exemplu ruta ocolește complet furtuna, după ocolire, bateria ajunge la 9% și pentru aterizare sigură sunt necesare minimum 15%. În acest caz, aeronava a rezolvat problema atmosferică, dar a creat una energetică. Din acest motiv, consider că energia este un criteriu de siguranță, nu doar un criteriu de optimizare.

Aș utiliza următoarea ordine a priorităților: menținerea zborului controlat, evitarea unui pericol atmosferic inacceptabil, menținerea sistemelor esențiale, menținerea

comunicației, dacă este posibil, continuarea misiunii și minimizarea întârzierii. În plus, aș introduce patru constrângeri care nu sunt compensate printr-un scor favorabil:

- Constrângere de pericol – ruta nu trebuie să traverseze o zonă clasificată ca având risc inacceptabil;
- Constrângere energetică - $E_{rămasă} \geq E_{aterizare} + E_{rezervă}$;
- Constrângere temporală – manevra trebuie să devină eficientă înainte de termenul calculat;
- Constrângere fizică – manevra trebuie să respecte limitele aeronavei.

Ipotezele utilizate

Pentru prototip fac următoarele presupuneri explicite: aeronaval utilizează propulsie electrică, un sistem de management al bateriei furnizează, aeronava își cunoaște poziția, viteza și direcția, furtunile severe nu trebuie traversate intenționat, aeronava modifică autonom traseul, există cel puțin o zonă sigură de aterizare și siguranța aeronavei are prioritate față de misiune. Aceste ipoteze nu sunt prezentate drept proprietăți reale ale unei aeronave pe Marte. Sunt condiții necesare pentru construirea prototipului și pot fi înlocuite când apar informații mai precise.

După această analiză, problema devine că sistemul trebuie să determine cât de fiabile sunt dovezile disponibile și apoi să selecteze continuu o acțiune care menține aeronava într-o stare recuperabilă, respectând constrângerile de risc, timp, energie și incertitudine.

Partea 2 – Soluția propusă

Conceptul

Propun sistemul autonom de siguranță în zbor bazat pe dovezi, risc și energie. Decizia este organizată în trei niveluri:

Fiabilitatea dovezilor

↓

Fezabilitatea din punct de vedere al siguranței

↓

Cea mai bună acțiune fezabilă

Sistemul nu optimizează direct o rută. Mai întâi verifică dacă interpretarea mediului este suficient de credibilă, apoi elimină acțiunile nesigure și doar la final clasifică alternativele rămase.

Arhitectura

Fluxul principal este senzori de mediu – manager de dovezi – estimator al furtunii – stare aeronavă + stare energetică – generator de acțiuni candidate – verificarea constrângerilor de siguranță – evaluarea acțiunilor fezabile – control zbor – reevaluare continuă. În paralel telemetrie – Pământ.

- Managerul de dovezi – primește datele senzorilor și verifică existența datelor, caută contradicții, calculează nivelul de încredere și clasifică starea drept CONFIRMAT, INCERT și CONFLICT. Un prototip simplu folosește $C = w_1 C_{cameră} + w_2 C_{particule} + w_3 C_{presiune} + w_4 C_{vânt}$. Ponderile și pragurile sunt parametri care trebuie testați, nu valori considerate automat corecte.

Furtuna este modelată prin centru, rază, severitate, direcție și incertitudine. Aeronava nu folosește doar raza estimată a furtunii, ci $R_{furtună} + R_{incertitudine} + R_{marjă}$. De exemplu raza estimată – 5 km, incertitudine – 2 km și marjă fizică – 1 km rezultă: $R_{sigur} = 8$ km. Dacă nivelul de încredere în predicție scade, marja de incertitudine crește. Date mai puțin sigure produc automat un comportament mai conservator.

Energia disponibilă este aproximată prin $E_{disponibilă} = C_{baterie} \times SOC$. Pentru fiecare rută $E_{rută} = P_{zbor} \times T_{rută}$. Ruta este fezabilă numai dacă $E_{marjă} \geq 0$. O ocolire care evită perfect furtuna este respinsă dacă nu mai lasă suficientă energie pentru o stare sigură. Într-o situație critică, sarcinile electrice neesențiale, precum anumite instrumente științifice, sunt oprite temporar pentru a păstra energia pentru propulsie, control și aterizare.

Pentru prima versiune folosesc doar cinci acțiuni, CONTINUARE, OCOLIRE_STÂNGA, OCOLIRE_DREAPTA, ÎNTOARCERE și ATERIZARE. Am ales intenționat un set redus deoarece este mai ușor de implementat, testat și explicat decât un algoritm complex care generează permanent un număr mare de traiectorii.

Logica deciziei

Fiecare rută este verificată mai întâi față de constrângerile stricte. O rută este respinsă dacă intersectează zona periculoasă SAU marja energetică < 0 SAU manevra nu este executată înainte de termen SAU depășește limitele aeronavei. Doar rutele rămase primesc un scor. Un scor simplificat este $J = w_r R + w_e E + w_t T + w_u U$, unde: R – risc, E – consum energetic, T – timp suplimentar, U – expunere la incertitudine. Riscul are cea mai mare pondere. Important este că scorul nu salvează o rută care a încălcat deja o condiție de siguranță.

Tabel 1. Exemplu de logica

Acțiune	Risc	SOC final	Timp suplimentar	Rezultat
Continuare	Critic	58%	0 min	Respinsă
Ocolire stânga	Scăzut	41%	+ 4 min	Fezabilă
Ocolire dreapta	Mediu	38%	+ 6 min	Fezabilă
Întoarcere	Scăzut	29%	+ 12 min	Fezabilă
Aterizare	Scăzut	52%	-	Fezabilă

În acest caz, CONTINUARE este eliminată chiar dacă este eficientă energetic, deoarece intersectează zona periculoasă. Dacă OCOLIRE_STÂNGA are cel mai bun scor dintre variantele fezabile, aceasta devine acțiunea selectată.

```
def alege_actiune(aeronava, furtuna, senzori):
    dovezi = evalueaza_dovezi(senzori)
    furtuna = actualizeaza_incetitudinea(furtuna, dovezi)
    termen = calculeaza_termen_decizie(aeronava, furtuna)
    rute = genereaza_rute_candidate(aeronava, furtuna)
    fezabile = []

    for ruta in rute:
        verifica_furtuna(ruta, furtuna)
        calculeaza_energie(ruta, aeronava)
        verifica_rezerva(ruta)
        verifica_timp(ruta, termen)
        verifica_limite_aeronava(ruta)

        if ruta.este_sigura():
            fezabile.append(ruta)

    if not fezabile:
        return "ATERIZARE_SIGURA"

    return min(fezabile, key=lambda ruta: ruta.scor)
```

Fig. 1 – Exemplu de pseudocod în Python

Reevaluare continuă

Decizia nu este definitivă. În timpul manevrei, sistemul repetă măsurare – estimare – verificarea dovezilor – actualizarea furtunii – reevaluarea rutelor – execuție. Dacă furtuna își schimbă direcția și consumul real este mai mare decât cel estimat, acțiunea este înlocuită. De exemplu inițial se poate OCOLIRE_STÂNGA. După 2 minute furtuna se deplasează spre stânga. Noua evaluare este că OCOLIRE_STÂNGA nu mai este sigură. Deci noua acțiune este ATERIZARE. Autonomia nu înseamnă întreruperea comunicației. Aeronava transmite observațiile, nivelul de încredere, poziția estimată a furtunii, starea bateriei, rutele analizate, motivele respingerii și acțiunea selectată. Dar nu așteaptă aprobarea înainte de executarea unei manevre critice. O comanda primită ulterior de pe Pământ trebuie verificată din nou față de situația actuală. Dacă este bazată pe o stare veche și nu mai respectă limitele de siguranță, aeronava nu o execută automat.

Pentru validare propun un simulator 2D realizat în Python cu dificultate intenționat moderată. Clasele principale sunt Aeronava, Baterie, Furtuna, Ruta, ManagerDovezi și ControlerAutonom.

- Aeronava – conține poziție, destinație, viteză, capacitatea bateriei, SOC și puterea estimată;
- Furtuna – conține centru, rază, incertitudine și severitate;
- Ruta – conține puncte, distanță, timp, energie necesară, SOC final, intersecție cu furtuna, scor și motiv de respingere.

Fluxul programului este semnale senzori – evaluare dovezi – calcul zonă de siguranță – generare rute – verificare geometrie – verificare energie – verificare timp – eliminare rute nesigure – selectare acțiune – actualizare stare. Pentru implementare aș utiliza Python 3, dataclasses, math, csv și matplotlib. Nu consider necesare Machine Learning, ROS și un simulator aerodinamic complex pentru a valida prima versiune a conceptului.

Aș testa cel puțin următoarele situații:

- Furtună severă confirmată – senzorii sunt de acord. Rezultat așteptat – ruta originală este respinsă și este selectată o ocolire fezabilă.
- Senzori contradictorii – camera indică pericol ridicat, dar ceilalți senzori indică valori normale. Rezultat așteptat – CONFLICT, încredere redusă și marjă de siguranță mai mare.

- Incertitudine mare și timp redus – informațiile nu sunt suficiente pentru confirmare, dar timpul disponibil se apropie de termenul minim. Rezultat așteptat – sistemul nu așteaptă certitudine perfectă, ci adoptă o variantă conservatoare.
- Ocolire posibilă, dar energie insuficientă – ruta evită furtuna, însă ar coborî bateria sub rezerva de siguranță. Rezultat așteptat – ocolirea este respinsă și se alege întoarcerea și aterizarea.
- Schimbarea furtunii – ruta aleasă inițial devine periculoasă. Rezultat așteptat – controlerul reevaluează situația și selectează altă stare sigură.

Validarea propriilor verificări

Pentru Managerul de Dovezi aş utiliza teste ale pragurilor, eliminarea pe rând a senzorilor, senzori blocați la valori extreme, date lipsă, valori contradictorii și teste în apropierea limitelor. Într-o etapă ulterioară aş utiliza simulări în care poziția furtunii, valorile senzorilor și consumul energetic sunt perturbate aleator. Scopul este să verific nu doar dacă sistemul produce o decizie, ci cât de robustă este decizia când datele nu sunt ideale.

Aş urmări intrarea sau nu în zona periculoasă, marja energetică disponibilă, timpul de decizie, distanța suplimentară, numărul de replanificări, conflictele între senzori și starea finală a aeronavei. Stările finale sunt MISIUNE_FINALIZATĂ, DEVIERE_SIGURĂ, ÎNTOARCERE, ATERIZARE_SIGURĂ, INTRARE_ÎN_FURTUNĂ și EPUIZARE_ENERGIE.

Pentru evaluare aş compara:

- Situația de bază – aeronava păstrează ruta inițială și așteaptă răspunsul Pământului.
- Soluția propusă – aeronava folosește decizia autonomă.

Tabel 2 – Comparație cu un sistem de bază

Indicator	Situație de bază	Soluție propusă
Așteaptă Pământul	Da	Nu
Decizie autonomă	Nu	Da
Verifică energia	Nu	Da
Gestionează incertitudinea	Nu	Da
Poate replanifica	Nu	Da

Valorile cantitative finale vor fi produse de simulator, nu introduse manual.

Alegeri tehnice și limitări

Am ales o logică deterministă în locul unui model ML pentru decizia finală deoarece sistemul are constrângeri fizice explicite și importante pentru siguranță. Machine Learning este util pentru detecția și predicția furtunii, dar rezultatul ar trebui să treacă în continuare prin reguli deterministe privind energia, timpul și limitele aeronavei. Am ales, de asemenea, rute candidate în locul optimizării continue, un model 2D în locul unui simulator atmosferic 3D, un model energetic simplificat și margini conservative pentru incertitudine. Aceste alegeri reduc realismul, dar fac logica ușor de testat și explicat. Prototipul nu modelează aerodinamica pe Marte în detaliu, efectele complete ale particulelor de praf, motoarele și invertoarele la nivel electromagnetic, electrochimia bateriei, terenul 3D și sisteme certificate. Scopul primei versiuni este validarea procesului de decizie, nu construirea unui simulator aerospațial complet.

Fidelitatea fizică poate fi crescută ulterior, după ce logica de bază a fost validată.

Utilizarea instrumentelor AI

Am utilizat ChatGPT în etapa de explorare pentru structurarea problemei, identificarea unor situații limită și analiza alternativelor.

Concluzii

Problema nu se reduce la alegerea unei direcții în jurul unei furtuni. Aeronava trebuie să decidă într-un mediu în care informațiile sunt incomplete, predicțiile sunt greșite, senzorii se contrazic, timpul este limitat, iar energia disponibilă trebuie conservată. Sistemul verifică mai întâi informația, apoi elimină acțiunile care încalcă limitele de siguranță și abia după aceea optimizează între alternative. Principul central este că aeronava trebuie să păstreze o stare sigură și recuperabilă înainte de a maximiza progresul misiunii. Prototipul Python este intenționat simplu. Scopul său este să facă vizibile ipotezele, incertitudinile, verificările și motivele fiecărei decizii, astfel încât acestea să fie testate și îmbunătățite iterativ.